wso2 | Executive Briefing

EXECUTIVE OVERVIEW

The DevOps OverSight Agent

An AI agent that diagnoses production incidents in minutes — across both your logging and monitoring platforms — and never acts without a human's sign-off.

Measured on a production-grade AI model

Scott Bechtel · For leadership review · July 2026

THE ONE-SLIDE VERSION

Executive summary

- **The problem:** engineers lose 20–40 minutes per major incident manually cross-referencing two separate tools before they even know what's wrong.
- **What we built:** an AI agent that does that cross-tool detective work automatically — then proposes one specific fix and waits for a human to approve it.
- **It never acts alone:** no fix runs, no system changes, without an explicit human approval. This is enforced, not just policy.
- **It's measured, not promised:** on a production-grade AI model, it correctly diagnosed the test incident 18 times out of 18.

100%
correct diagnosis rate (measured)

14–36s
time to a proposed fix

0
vendor lock-in

◆ **Bottom line**

This isn't a concept — it's a working system with measured, repeatable results.

THE BUSINESS PROBLEM

The cost of manually correlating two tools

- **Two systems, no shared language.** Your logs live in one platform, your metrics and traces in another. They don't talk to each other.
- **A person is the integration.** During an incident, an engineer manually copies identifiers between tools, cross-referencing by hand and by memory.
- **It doesn't scale.** The bigger and more complex your systems get, the worse this gets — and it depends on whoever happens to be on call.
- **The stakes are real.** Every minute of manual correlation is a minute of extended downtime, customer impact, and risk exposure.

20–40 min

lost per major incident to manual correlation alone

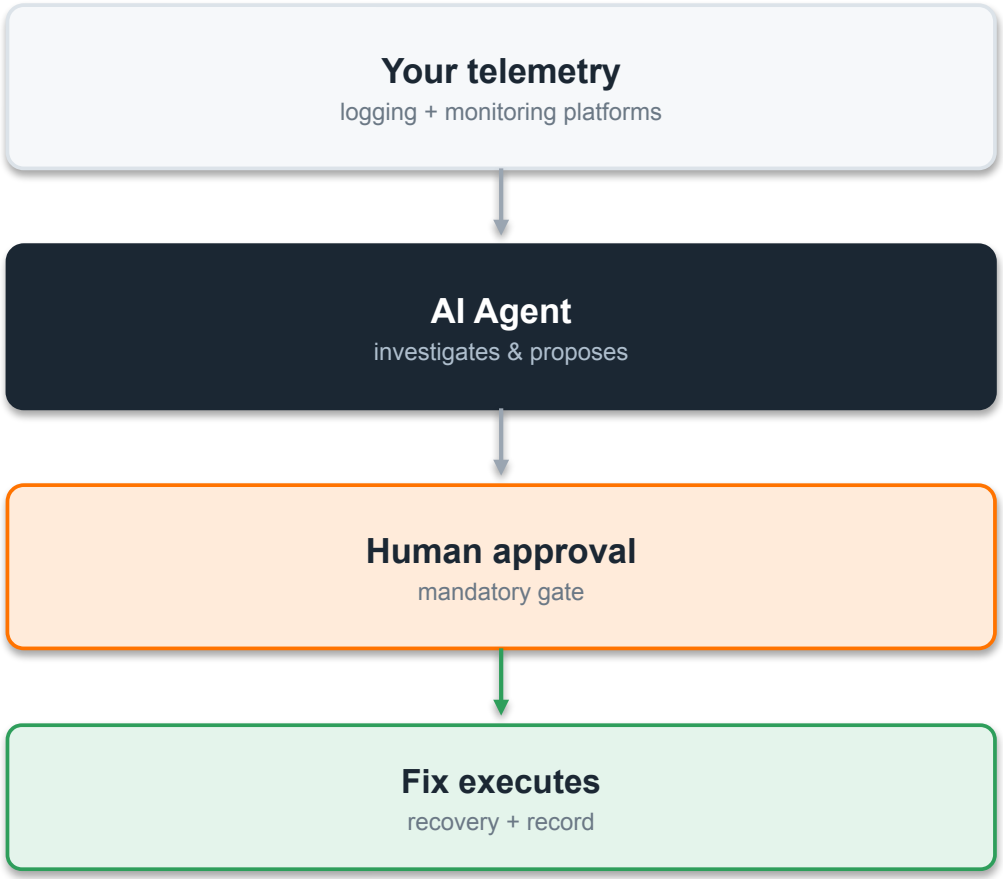
◆ **Why it matters at your level**

This is a direct, measurable line item in mean-time-to-resolution — and MTTR is a direct line item in customer trust and operational cost.

INTRODUCTION

What it is, in plain terms

- **It watches.** The agent continuously has access to both your logging and monitoring platforms — the same data your engineers already look at.
- **It investigates automatically.** When something breaks, it pulls evidence from both tools and connects them using the identifier that already links them together.
- **It proposes — it doesn't act.** It names one specific fix from a pre-approved list. It never invents an action and never runs one unsupervised.
- **A human always decides.** The fix only executes after an engineer explicitly approves it. Every action is logged.



HOW IT WORKS

The workflow cycle

1

Detect

An alert fires — an anomaly in monitoring, or an incoming alert.

2

Investigate

The agent pulls evidence from both platforms and links it by a shared identifier.

3

Diagnose

Root cause is identified, and the agent assesses what else is affected.

4

Suggest a runbook

The agent proposes ONE specific, named fix — never vague advice.

5

Approve

A human reviews the evidence and the proposed fix, then explicitly approves.

6

Remediate & recover

The fix runs, systems recover, and a record is generated automatically.

TWO WAYS WE BUILT IT

We built it twice, to compare

Same capability, same test scenario — two different engineering approaches, so we could evaluate real trade-offs instead of guessing.

SOLUTION 1

Ballerina + MCP Proxy

One agent, one gateway to both platforms. Built to run natively on our WSO2 Agent Manager platform.

SOLUTION 2

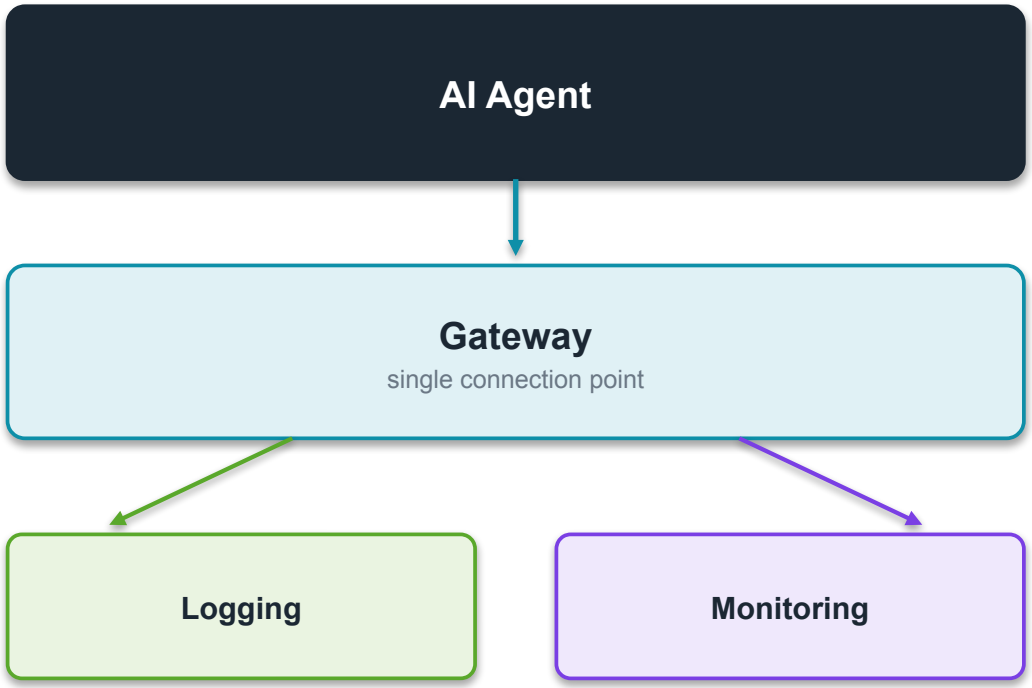
LangChain + A2A

Specialist agents divide the work, one per platform. Built in Python — the most widely adopted AI agent ecosystem.

SOLUTION 1

Ballerina + MCP Proxy

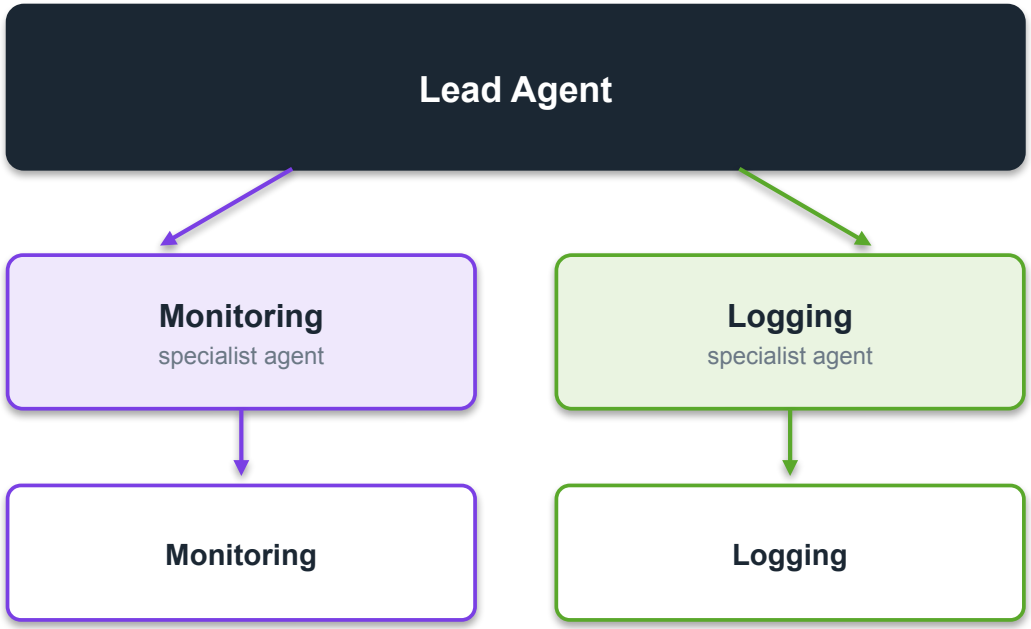
- **One agent, one gateway.** A single AI agent connects through one gateway that reaches both your logging and monitoring platforms.
- **Fewer moving parts.** Fewer components to operate, secure, and audit — a simpler system to run day to day.
- **Native to our platform.** Runs directly on WSO2's own Agent Manager, our platform for governing and observing AI agents.
- **One consistent language.** The entire stack — agent, gateway, and services — is built in a single language end to end.



SOLUTION 2

LangChain + A2A

- **Specialist agents.** A lead agent delegates to two specialist agents, one dedicated to each platform.
- **Built on Python.** Uses LangChain, the most widely adopted framework for building AI agents — a large hiring and support pool.
- **Approval enforced in code.** The human-approval step is built into the system's control flow, not just an instruction to the AI.
- **Scales to many platforms.** This division-of-labor pattern extends cleanly if more tools or teams are added later.



MEASURED RESULTS

What we actually measured

Metric — measured on a production-grade AI model	Ballerina + Proxy	LangChain + A2A
Correct diagnosis rate	**100% (18 / 18 test runs)**	**100% (18 / 18 test runs)**
Median time to a proposed fix	**~14 seconds**	~36 seconds
Test runs evaluated	18 (3 independent batches of 6)	18 (3 independent batches of 6)

◆ **Why this matters**

Both are reliable enough to trust in a live environment. Ballerina resolves faster in this test because it makes fewer hops between agents for the same investigation.

◆ **A critical cost finding**

A free, locally-hosted AI model is possible for either approach, but was only 28–61% reliable in the same test. Reliability came from using a production-grade model, not from which architecture we chose — worth weighing directly against licensing cost.

Method: identical simulated incident injected into a test service; timed end-to-end from alert to a proposed fix; 3 independent test batches of 6 runs per approach.

TRUST & GOVERNANCE

Nothing changes without a human

1

Propose, never act alone

The agent names a fix and stops. It cannot execute a change without an explicit, recorded human approval.

2

A bounded set of actions

The agent can only ever propose from a fixed, pre-approved list of fixes — never an arbitrary command.

3

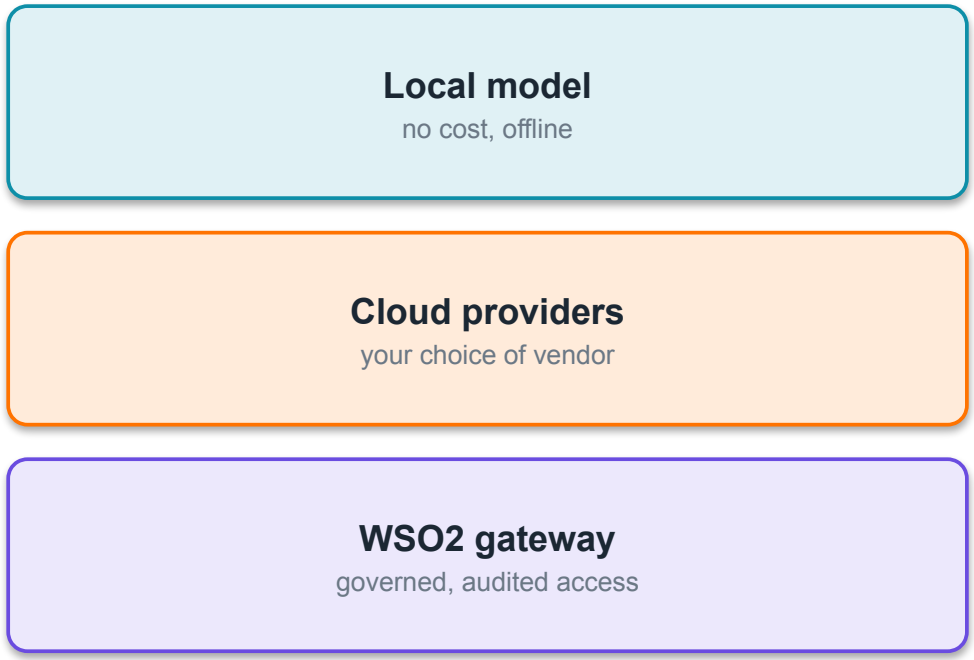
Every action is logged

Every proposal, approval, and executed action is recorded in an audit trail.

FLEXIBILITY

No AI vendor lock-in

- **The AI model is a setting, not a commitment.** Switch between AI providers, or run entirely on your own infrastructure, without changing the system.
- **Works fully offline if required.** A locally-hosted, no-cost model option exists for environments that can't use an outside AI vendor.
- **Same investigation, any provider.** We've proven the same investigation completes successfully across multiple AI providers with no code changes.



PROOF, NOT PROMISES

A real 5-minute demonstration

1

Inject

A real fault is introduced into a test service.

2

Detect & investigate

The agent notices and automatically investigates across both platforms.

3

Propose & approve

It proposes a specific fix; an operator approves it.

4

Recover

The fix runs and the system returns to healthy — automatically documented.

◆ **Runs on a laptop, no live accounts required**

This can be demonstrated live, end to end, in under five minutes — including for a client, on the spot.

WHERE THINGS STAND

Our read, and what would decide it

Both approaches are production-viable on a real AI model. Ballerina measured faster; LangChain offers a code-enforced approval gate and a larger talent pool.

Favors Ballerina + Proxy

- Standardizing on our own WSO2 platform.
- Fewest components to run and secure.
- Faster measured response time.

Favors LangChain + A2A

- Teams already build in Python.
- Approval gate enforced in code, by design.
- Broadest hiring and support pool.

NEXT STEPS

Path to a production pilot

1. Choose a direction — or run a short pilot of both against a real, low-risk service.
2. Connect it to your live logging and monitoring platforms (no code changes required).
3. Finalize the approval workflow and audit requirements with your compliance team.
4. Run a live, supervised pilot on a non-critical service before wider rollout.

wso2

Thank you.

Happy to go deeper on any part of this — architecture, results, or rollout planning.

Scott Bechtel · WSO2 Forward Deployed Engineering